This slide was sponsored by stress, adrenaline, and an unhealthy amount of cortisol.

PRODUCTION INCIDENTS

This slide was sponsored by stress, adrenaline, and an unhealthy amount of cortisol.

@nairi.dev

NAIRI / NAIRI.DEV / @NAIRIHAR

- 🥷 I work as a **Tech Lead**
- 🎤 **Speaker,** Conference organiser
- 💛 Creator of the **JavaScript Armenia** Community
- ✍️ Writing about **JavaScript** and related technologies
- 🎙️ Hosting **podcasts** for the Armenian tech community
- 🔥 **Fancy** username holder on **Telegram: @javascript**

You Don't Know Node.js EventLoop

@nairihar.medium.com

AGENDA

Intro: Life Behind Production Incidents

- ***My Story**: Challenges, Failures*

Retry Mechanism: Making Failures Recoverable

- ***Kafka**: Reliable Retries & Message Recovery*
- ***KafkaUI**: Monitoring and Management*

***Metrics & Alerts**: Grafana & PagerDuty*

***CAP Theorem:** Distributed Data Store Theorem*

***Feature Flags:** Taking Control of Prod*

- *Unleash: Safer Releases & Faster Recovery*

System Lighthouses

- *Checking service health and taking action.*

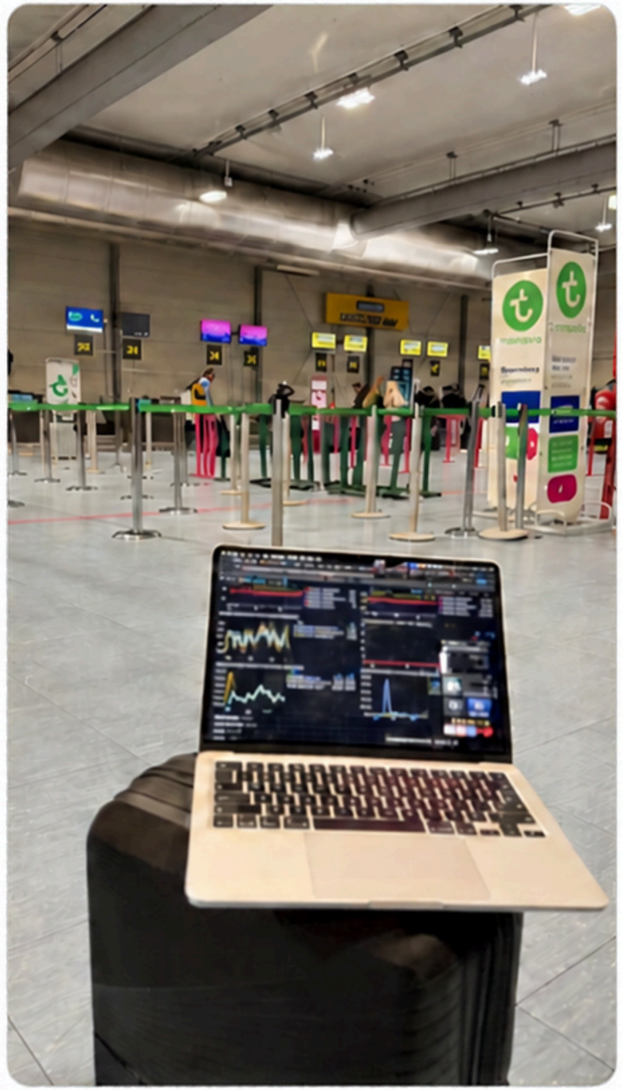
Prod Manual Activity

- ***Slack:** Being aware of who is doing what*

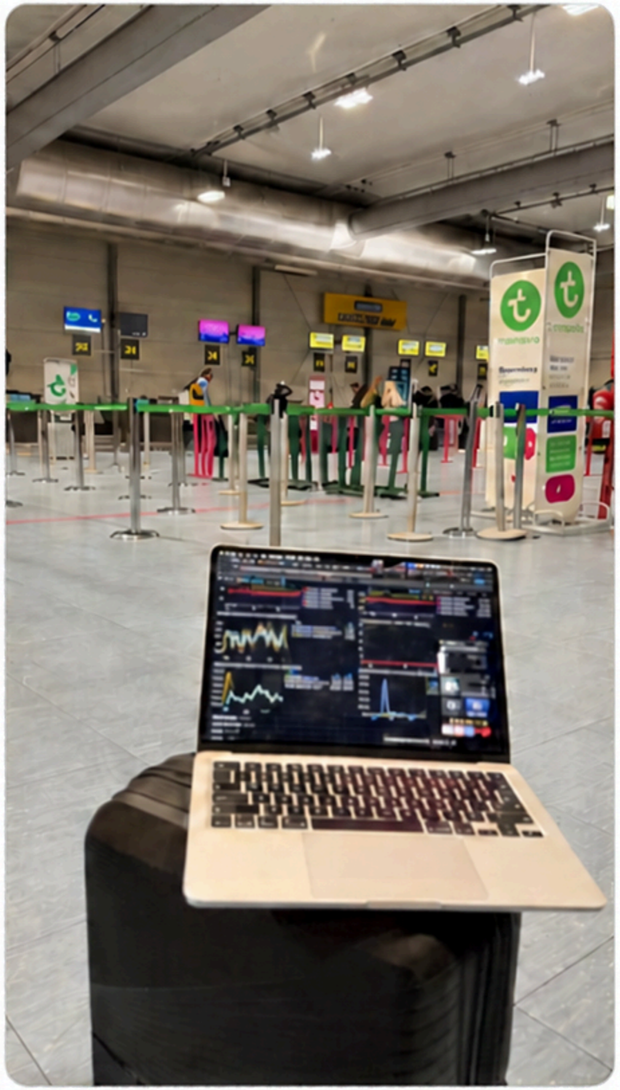
Have you ever had any
production incidents?

The story ...

The most **inconvenient places**
where I’ve had production incidents.



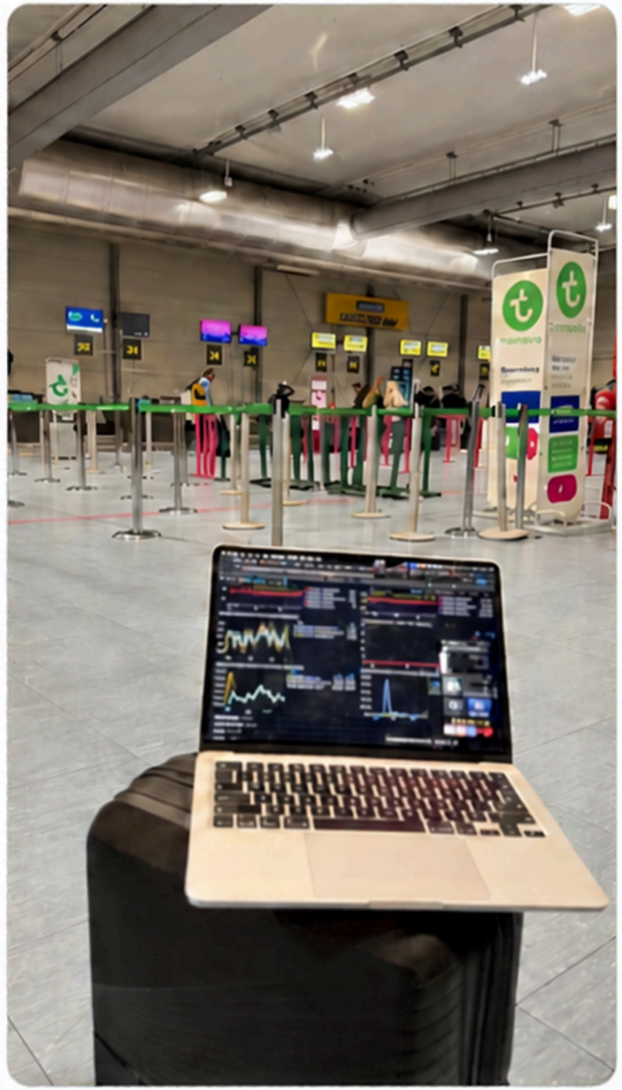
Airport



Airport

Hospital

@nairihar



Airport

Hospital

Wedding Party

@nairihar

@nairi.dev

Am I late?
@nairi.dev

Burnout...

WHAT MORE DO YOU WANT FROM ME?

We build tools to help ourselves

We build tools to help ourselves

The product team wouldn’t understand, so we just built it behind the scenes.

We build tools to help ourselves

The product team wouldn’t understand, so we just built it behind the scenes.

But technically speaking...

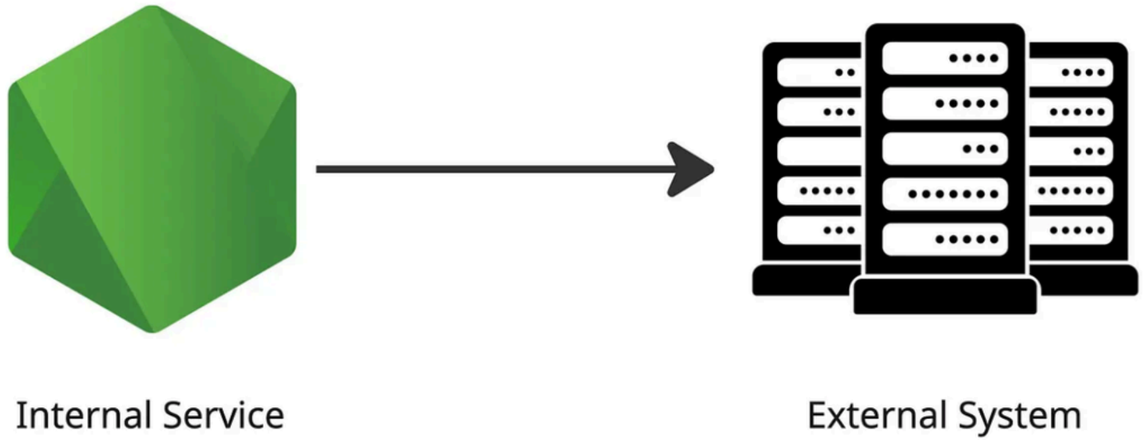
What can we do?

Simple Example

Basic flow: An Internal Service sends an event to an External System.

Simple Example

Basic flow: An Internal Service sends an event to an External System.



Basic Retry Mechanism

Very basic retry mechanism that tries to run a function

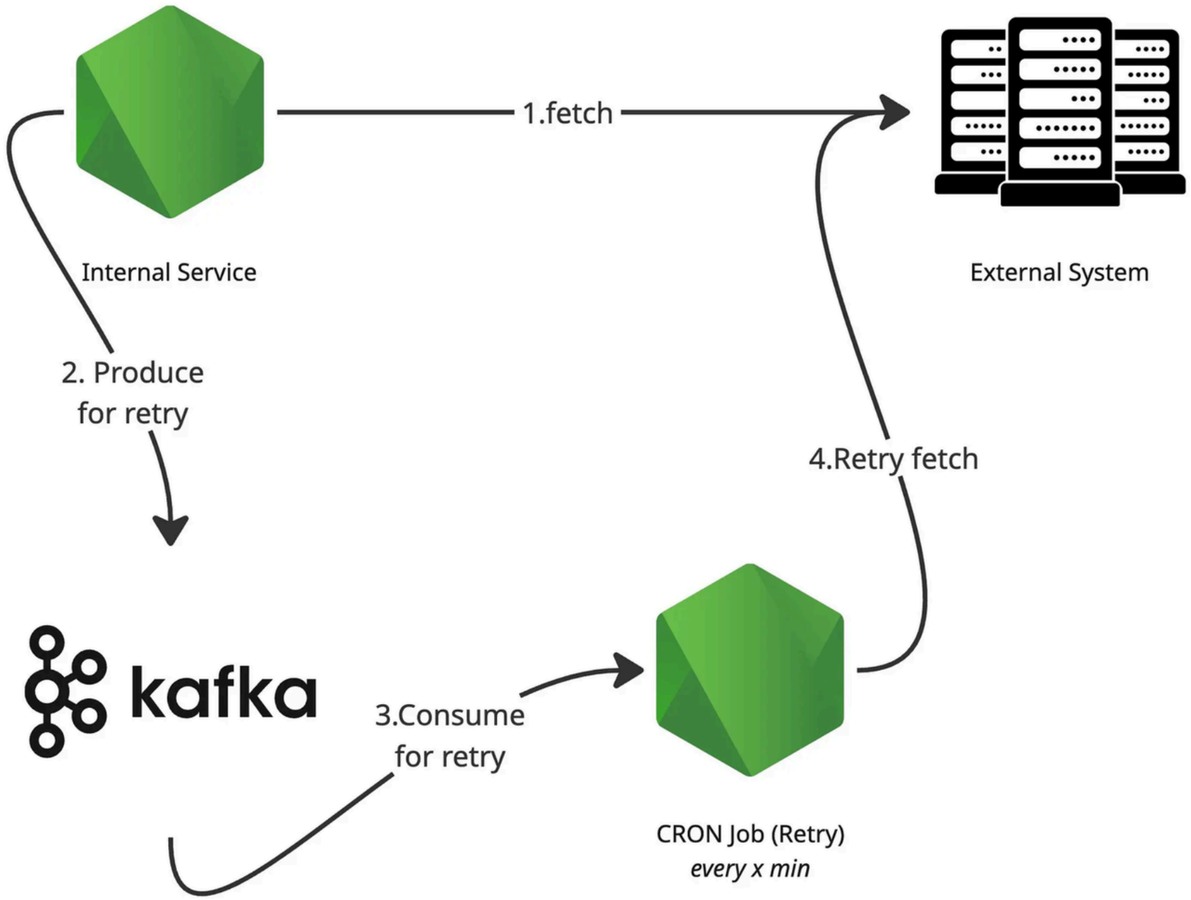
```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function retry(fn, retries = 3, delayMs = 3000) {
  let lastError;

  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      lastError = err;

      if (attempt === retries) {
        throw lastError; // final fail
      }

      console.log(`Attempt ${attempt} failed. Retrying in ${delayMs}ms...`);
      await sleep(delayMs);
    }
  }
}
```

Kafka as a Solution (v1)

First try if fails move the message to Kafka for the retry

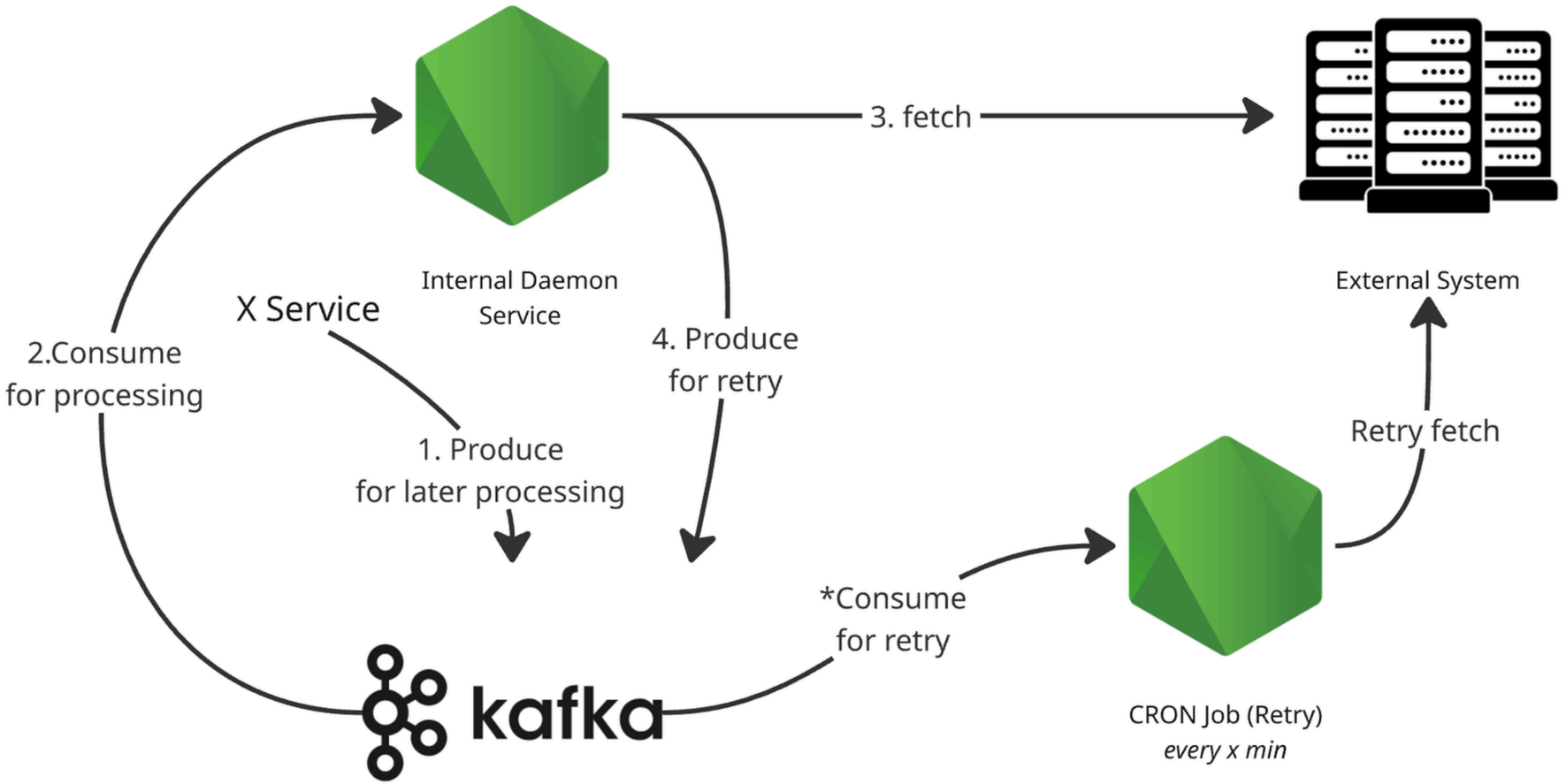
Kafka as a Solution (v1)

First try if fails move the message to Kafka for the retry



Kafka as a Solution (v2)

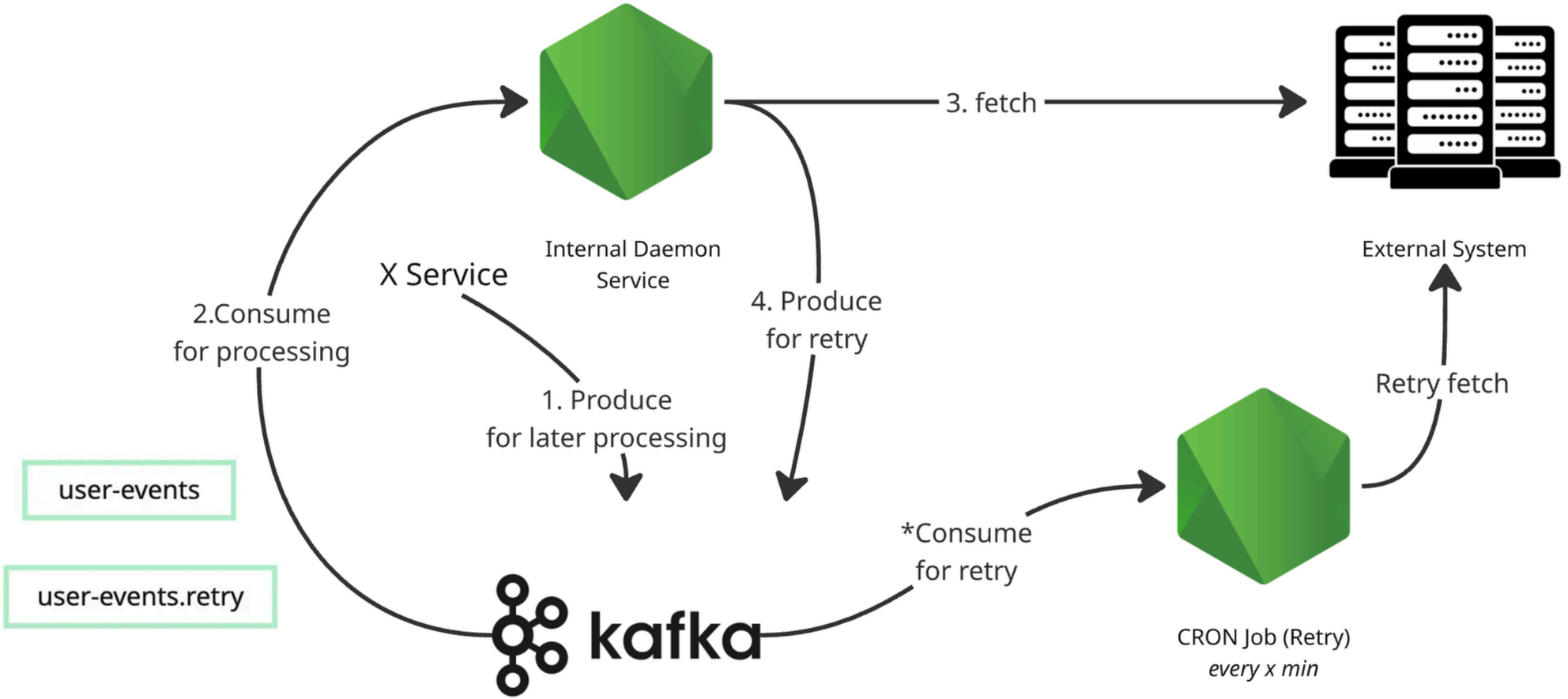
First move the message to Kafka then start the processing



@nairihar

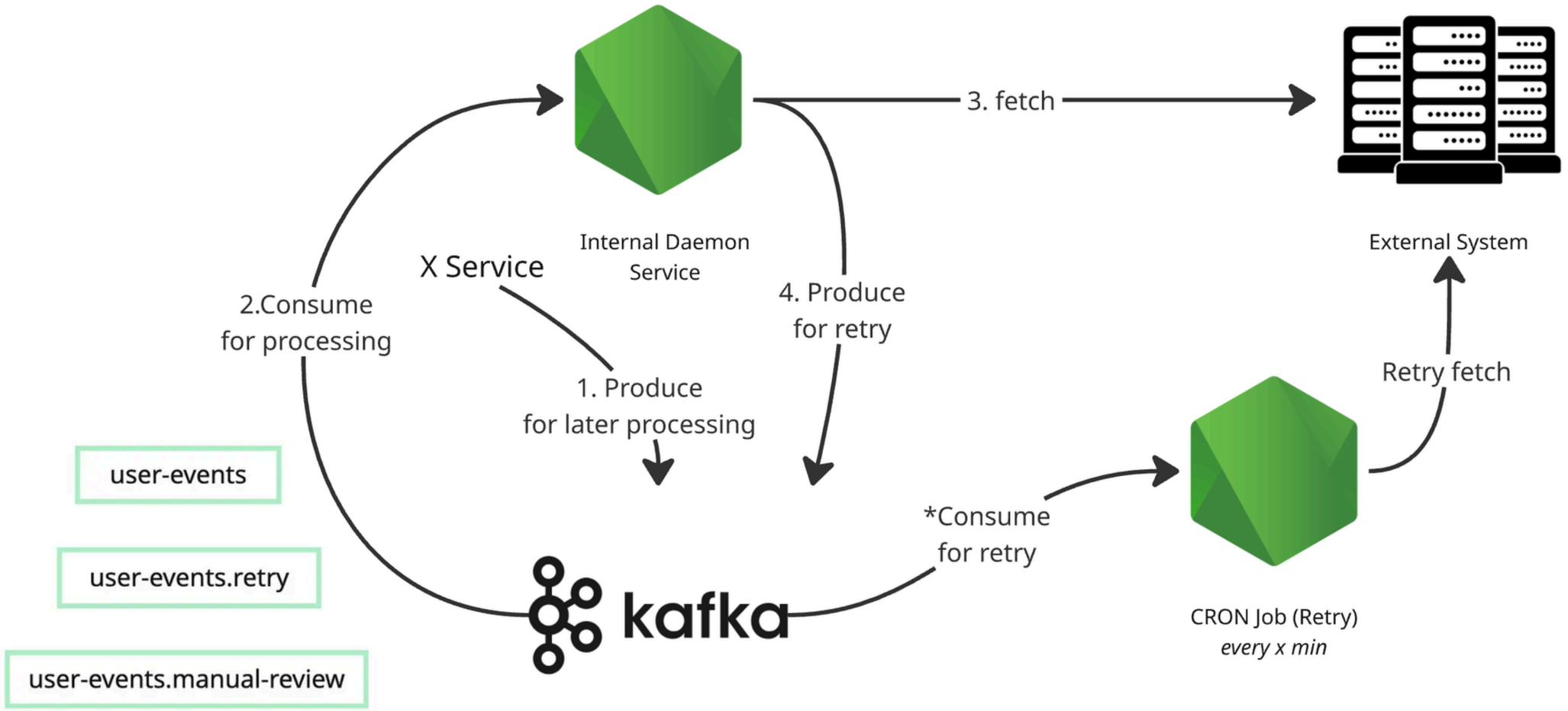
Kafka as a Solution (v2)

First move the message to Kafka then start the processing



Kafka as a Solution (v2)

First move the message to Kafka then start the processing



Three-Topic Kafka Retry Architecture

Reliable event processing with retries and failure recovery.

Three-Topic Kafka Retry Architecture

Reliable event processing with retries and failure recovery.

user-events

Stores incoming events for immediate processing.

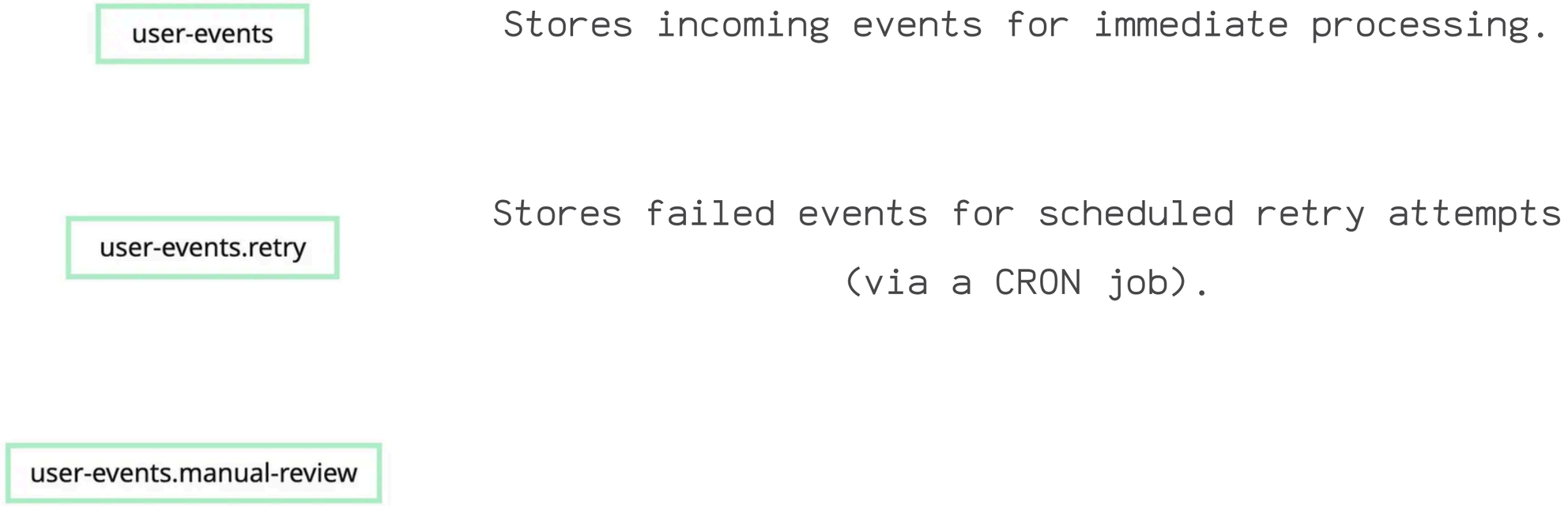
user-events.retry

user-events.manual-review

@nairihar

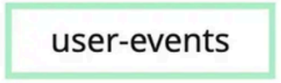
Three-Topic Kafka Retry Architecture

Reliable event processing with retries and failure recovery.

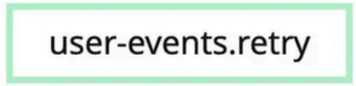


Three-Topic Kafka Retry Architecture

Reliable event processing with retries and failure recovery.

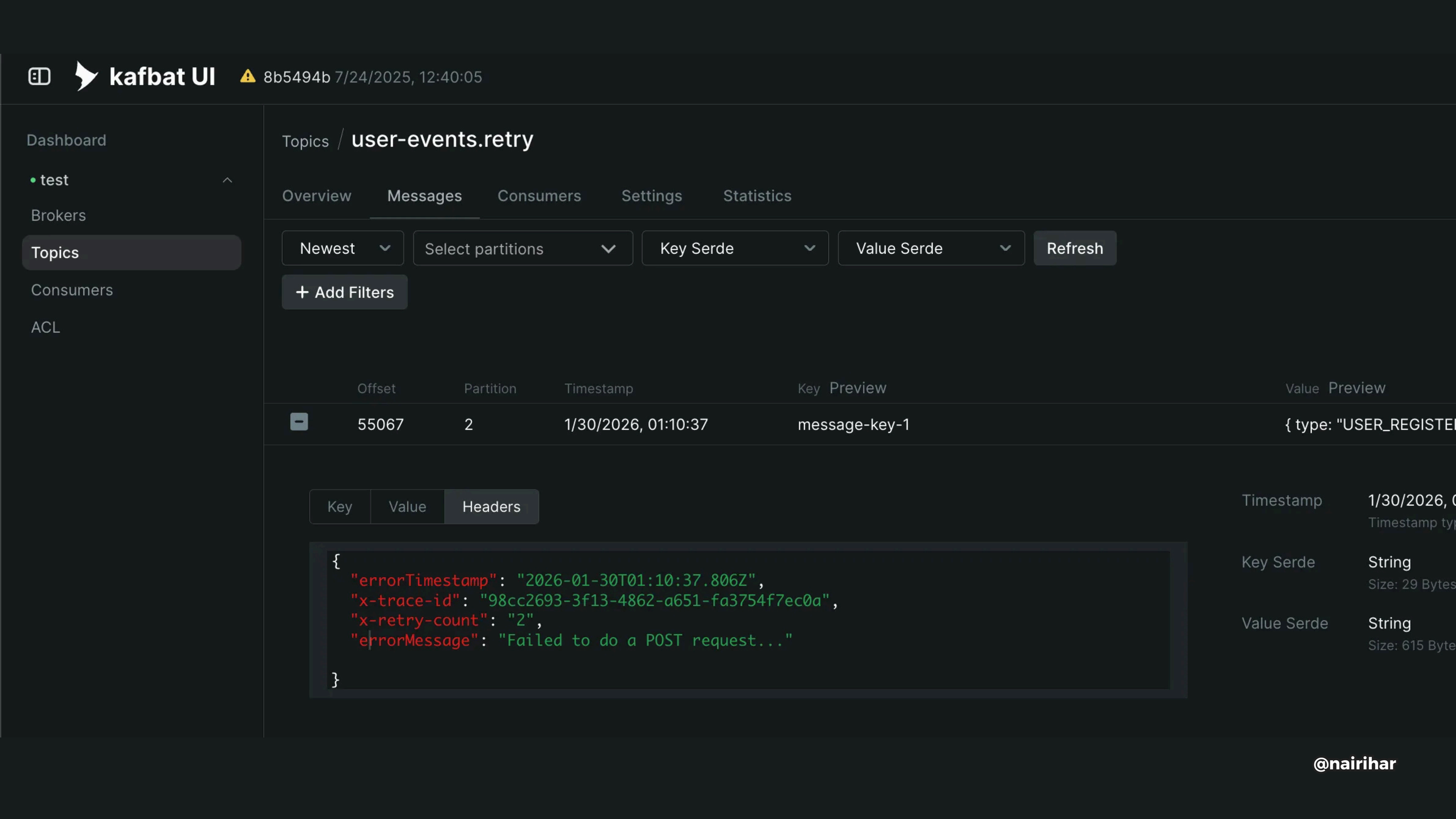


Stores incoming events for immediate processing.



Stores failed events for scheduled retry attempts (via a CRON job).

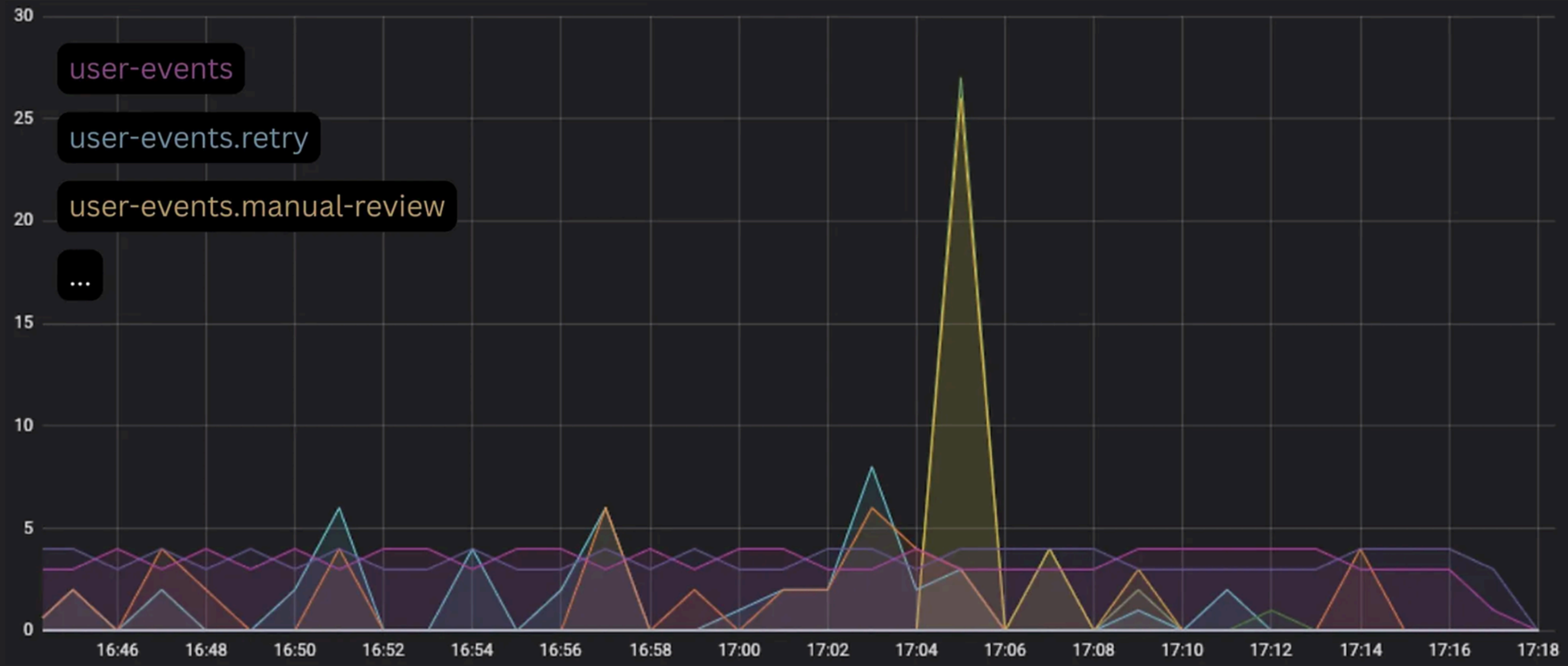
Stores events that have failed all retry attempts and require investigation or manual reprocessing.


kafbat UI
8b5494b 7/24/2025, 12:40:05
Dashboard
test
Brokers
Topics
Consumers
ACL
Topics / user-events.retry
Overview
Messages
Consumers
Settings
Statistics
Newest
Select partitions
Key Serde
Value Serde
Refresh
+ Add Filters
Offset
Partition
Timestamp
Key Preview
Value Preview
55067
2
1/30/2026, 01:10:37
message-key-1
{ type: "USER_REGISTE
Key
Value
Headers
{
"errorTimestamp": "2026-01-30T01:10:37.806Z",
"x-trace-id": "98cc2693-3f13-4862-a651-fa3754f7ec0a",
"x-retry-count": "2",
"errorMessage": "Failed to do a POST request..."
}
Timestamp
1/30/2026,
Timestamp typ
Key Serde
String
Size: 29 Bytes
Value Serde
String
Size: 615 Byte

@nairihar

Monitoring & Alerting: Grafana + PagerDuty

Real-time monitoring of Kafka failures with automated alerts for faster incident detection and response.



Kafka Message Recovery via CLI

Replay failed messages from the **manual-review** topic to the **retry** topic using a simple Kafka **command**, enabling reprocessing after fixing production issues.

Kafka Message Recovery via CLI

Replay failed messages from the **manual-review** topic to the **retry** topic using a simple Kafka **command**, enabling reprocessing after fixing production issues.

```
kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic source-topic \
  --group move-to-destination-v1 \
  --property print.key=true \
  --property key.separator=":" | \
kafka-console-producer --bootstrap-server localhost:9092 \
  --topic destination-topic \
  --property parse.key=true \
  --property key.separator=":"
```

Kafka Message Recovery via CLI

Replay failed messages from the **manual-review** topic to the **retry** topic using a simple Kafka **command**, enabling reprocessing after fixing production issues.

```
kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic source-topic \
  --group move-to-destination-v1 \
  --property print.key=true \
  --property key.separator=":" | \
kafka-console-producer --bootstrap-server localhost:9092 \
  --topic destination-topic \
  --property parse.key=true \
  --property key.separator=":"
```

user-events.manual-review

Kafka Message Recovery via CLI

Replay failed messages from the **manual-review** topic to the **retry** topic using a simple Kafka **command**, enabling reprocessing after fixing production issues.

```
kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic source-topic \
  --group move-to-destination-v1 \
  --property print.key=true \
  --property key.separator=":" | \
kafka-console-producer --bootstrap-server localhost:9092 \
  --topic destination-topic \
  --property parse.key=true \
  --property key.separator=":"
```

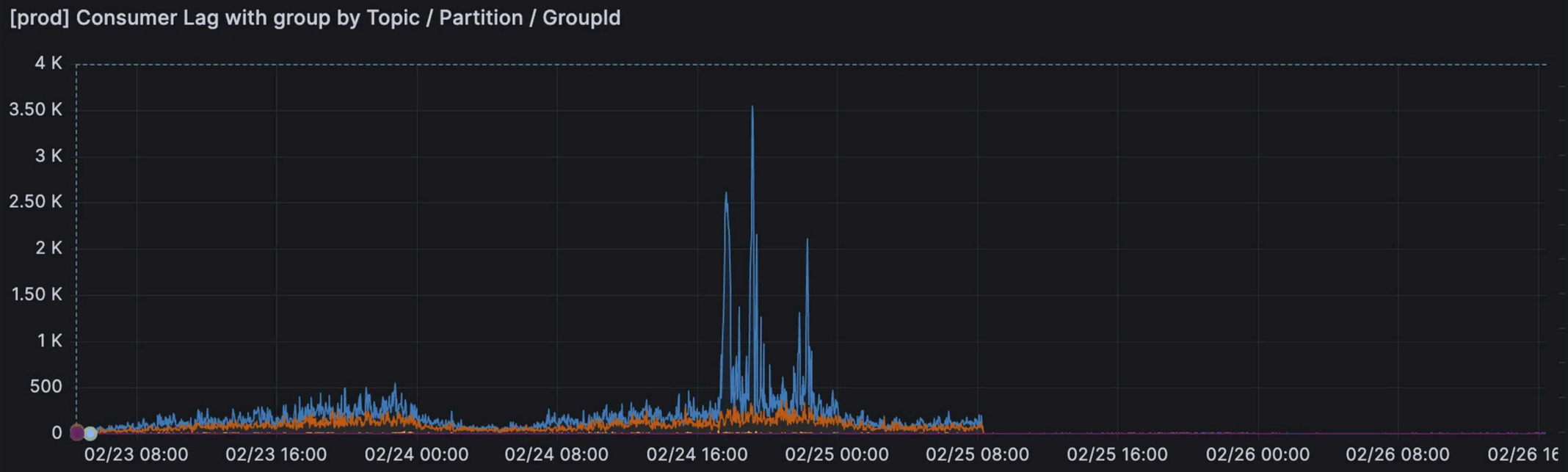
Kafka Message Recovery via CLI

Replay failed messages from the **manual-review** topic to the **retry** topic using a simple Kafka **command**, enabling reprocessing after fixing production issues.

```
kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic source-topic \
  --group move-to-destination-v1 \
  --property print.key=true \
  --property key.separator=":" | \
kafka-console-producer --bootstrap-server localhost:9092 \
  --topic destination-topic \
  --property parse.key=true \
  --property key.separator=":"
```

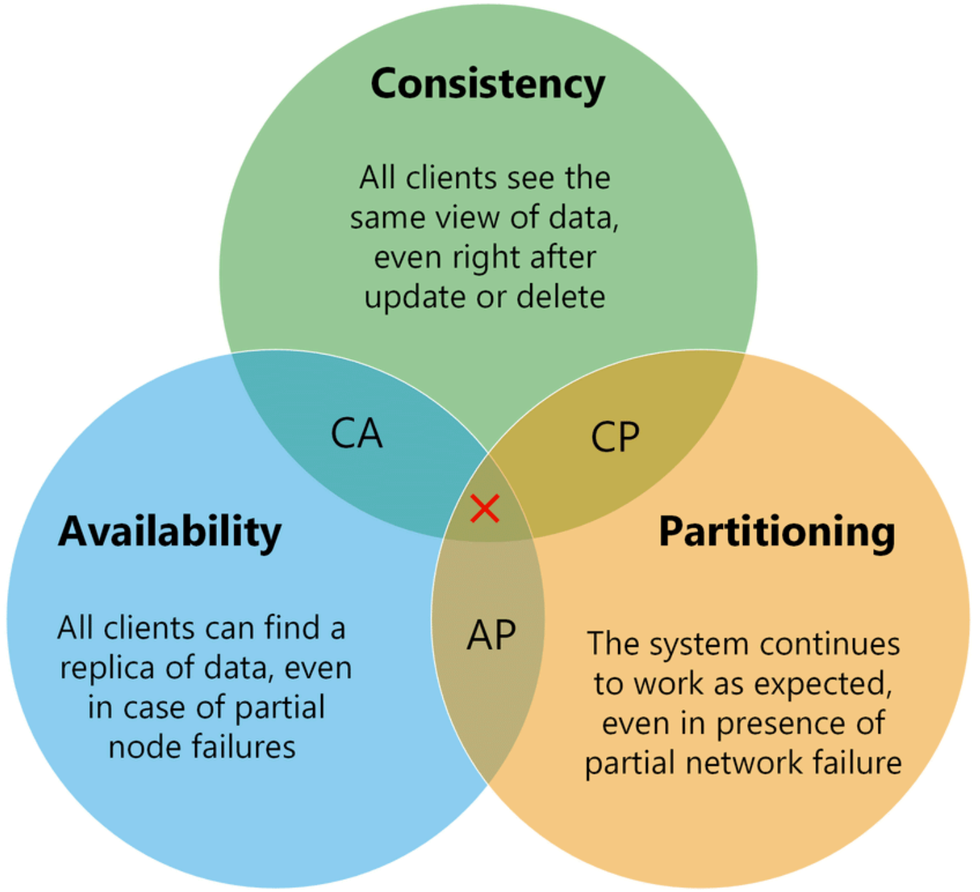
Kafka Consumer Lag in Production

Grafana shows lag spikes reaching 3,500 messages, followed by a drop to near zero after the release/fix.



Magic Trick
(Break)

CAP Theorem

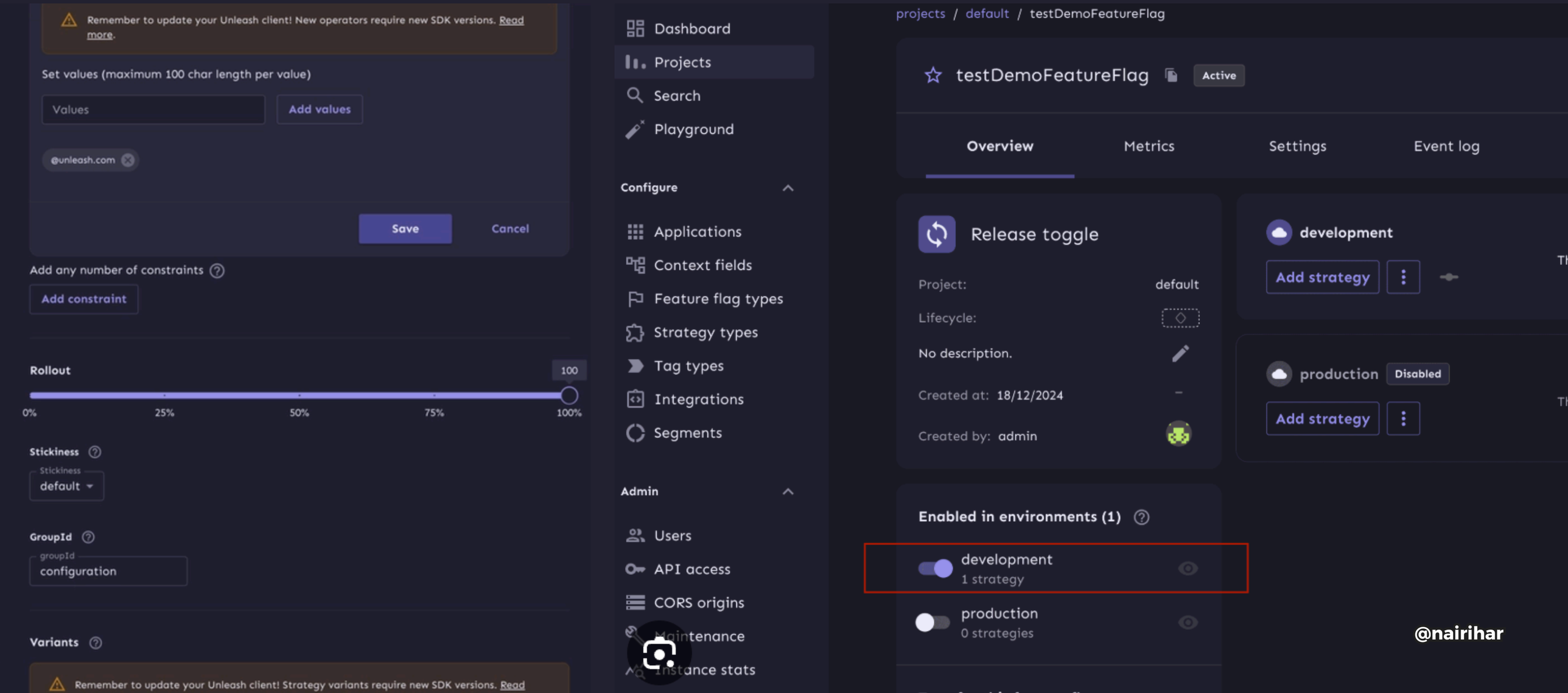


CAP Theorem

CAP theorem says a distributed system can fully guarantee
only two of these three at the same time: **Consistency, Availability,** and **Partition Tolerance.**

@nairihar

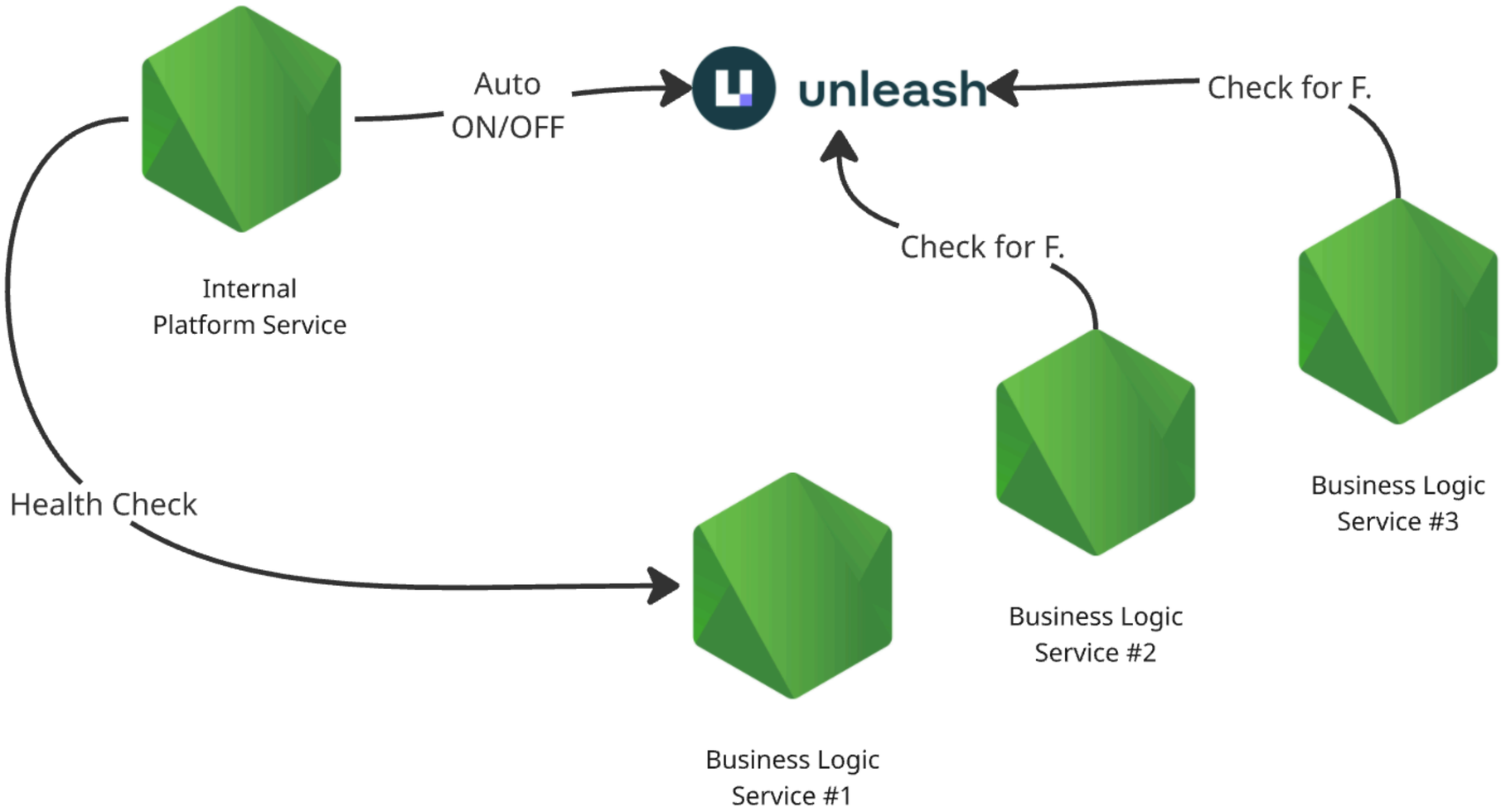
Feature Flags: manual switch



@nairihar

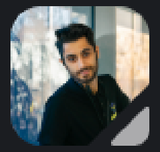
Feature Flags: automatic switch

Feature Flags: automatic switch



Prod Manual Activity

Prod Manual Activity

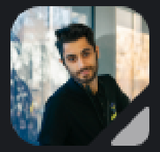


Nairi Harutyunyan

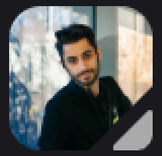
Whitelisting `69.183.14.120` in `system-prod-eu-north-0` cluster

@nairihar

Prod Manual Activity



Nairi Harutyunyan
Whitelisting `69.183.14.120` in `system-prod-eu-north-0` cluster



Nairi Harutyunyan
Adding `AUTO_SYNC` env in `connector-daemon` service

Slack channel ideas

Slack channel ideas

- *system-alerts*
- *system-alerts-nodejs*
- **system-swat*
- *system-deploys-prod*
- *system-deploys-non-prod*
- *manual-activity-prod*

SUMMARY...

Each syste/business is **unique...**

CAP Theorem

Eventual Consistency
(Kafka, Kafka UI)

Being aware of things happening

(Alerts, Slack Channels)

Being able to control things easily

(Feature Flags)

During incidents, **avoid long discussions**. Focus on **taking action** and **minimising** the scope for unnecessary debate.

THANK YOU!

@nairi.dev

@nairi.dev